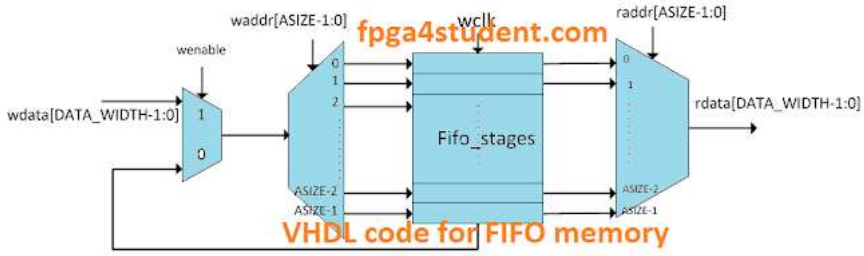
VHDL code for FIFO Memory

As requested by some readers, I made the VHDL code for the FIFO memory in this VHDL project(Verilog code for FIFO memory).

The FIFO has 16 8-bit data-width stages and five status signals including overflow, underflow, empty, full and threshold. The VHDL code for the FIFO memory is verified by the same Verilog testbench code by doing a mixed language simulation on Xilinx ISIM.



VHDL code for the FIFO as follows:

```
Library IEEE;
USE IEEE.Std_logic_1164.all;
-- FPGA projects using VHDL/ VHDL
-- fpga4student.com
-- VHDL code for FIFO memory
entity fifo_mem is
   port(
      data_out : out std_logic_vector(7 downto 0);
      fifo_full, fifo_empty, fifo_threshold,
      fifo_overflow, fifo_underflow: out std_logic;
      clk :in std_logic;
      rst_n: in std_logic;
      wr :in  std_logic;
      rd: in std_logic;
      data_in: in std_logic_vector(7 downto 0)
   );
end fifo_mem;
architecture Behavioral of fifo_mem is
component write_pointer
   port(
      wptr : out std_logic_vector(4 downto 0);
   fifo_we: out std_logic;
      clk :in std_logic;
   rst_n: in std_logic;
      wr :in  std_logic;
   fifo_full: in std_logic
   );
end component;
component read_pointer
   port(
      rptr : out std_logic_vector(4 downto 0);
   fifo_rd: out std_logic;
      clk :in std_logic;
   rst_n: in std_logic;
      rd :in  std_logic;
   fifo_empty: in std_logic
   );
end component;
component memory_array
   port(
```

```
      data_out : out std_logic_vector(7 downto 0);
   rptr: in  std_logic_vector(4 downto 0);
      clk :in std_logic;
   fifo_we: in std_logic;
      wptr :in  std_logic_vector(4 downto 0);
   data_in: in std_logic_vector(7 downto 0)
   );
end component;
component status_signal
   port(
      fifo_full, fifo_empty, fifo_threshold: out std_logic;
   fifo_overflow, fifo_underflow : out std_logic;
      wr, rd, fifo_we, fifo_rd,clk,rst_n :in std_logic;
      wptr, rptr: in  std_logic_vector(4 downto 0)
   );
end component;
  signal empty, full: std_logic;
  signal wptr,rptr: std_logic_vector(4 downto 0);
  signal fifo_we,fifo_rd: std_logic;
begin

 write_pointer_unit: write_pointer port map
      (
          wptr => wptr,
       fifo_we=> fifo_we,
       wr=> wr,
       fifo_full => full,
       clk => clk,
       rst_n => rst_n
      );
 read_pointer_unit: read_pointer port map
      (
       rptr => rptr,
       fifo_rd => fifo_rd,
       rd => rd ,
       fifo_empty => empty,
       clk => clk,
       rst_n => rst_n
      );
 memory_array_unit: memory_array port map
      (
       data_out => data_out,
       data_in => data_in,
       clk => clk,
       fifo_we => fifo_we,
       wptr => wptr,
       rptr => rptr
      );
 status_signal_unit: status_signal port map
      (
       fifo_full => full,
       fifo_empty => empty,
       fifo_threshold => fifo_threshold,
       fifo_overflow => fifo_overflow,
       fifo_underflow => fifo_underflow,
       wr => wr,
       rd => rd,
       fifo_we => fifo_we,
       fifo_rd => fifo_rd,
       wptr => wptr,
       rptr => rptr,
       clk => clk,
       rst_n => rst_n
```

```
      );
 fifo_empty <= empty;
 fifo_full <= full;
end Behavioral;
Library IEEE;
USE IEEE.Std_logic_1164.all;
USE ieee.std_logic_arith.all;
USE ieee.std_logic_unsigned.all;
-- FPGA projects using VHDL/ VHDL
-- fpga4student.com
-- VHDL code for FIFO memory
-- status signals
entity status_signal is
   port(
      fifo_full, fifo_empty, fifo_threshold: out std_logic;
   fifo_overflow, fifo_underflow : out std_logic;
      wr, rd, fifo_we, fifo_rd,clk,rst_n :in std_logic;
      wptr, rptr: in  std_logic_vector(4 downto 0)
   );
end status_signal;
architecture Behavioral of status_signal is
  signal fbit_comp, overflow_set, underflow_set: std_logic;
  signal pointer_equal: std_logic;
  signal pointer_result:std_logic_vector(4 downto 0);
  signal full, empty: std_logic;
begin

  fbit_comp <= wptr(4) xor rptr(4);
  pointer_equal <= '1'
  when (wptr(3 downto 0) = rptr(3 downto 0)) else '0';
  pointer_result <= wptr - rptr;
  overflow_set <= full and wr;
  underflow_set <= empty and rd;
  full <= fbit_comp and  pointer_equal;
  empty <= (not fbit_comp) and  pointer_equal;
  fifo_threshold <=  '1'
  when (pointer_result(4) or pointer_result(3))='1' else '0';
  fifo_full <= full;
  fifo_empty <= empty;
  process(clk,rst_n)
  begin
  if(rst_n='0') then
 fifo_overflow <= '0';
  elsif(rising_edge(clk)) then
     if ((overflow_set='1')and (fifo_rd='0')) then
  fifo_overflow <='1';
  elsif(fifo_rd='1') then
  fifo_overflow <= '0';
  end if;
  end if;
  end process;

  process(clk,rst_n)
  begin
  if(rst_n='0') then
 fifo_underflow <='0';
  elsif(rising_edge(clk)) then
    if((underflow_set='1')and(fifo_we='0')) then
       fifo_underflow <='1';
    elsif(fifo_we='1') then
      fifo_underflow <='0';
 end if;
   end if;
```

```
  end process;
end Behavioral;

Library IEEE;
USE IEEE.Std_logic_1164.all;
USE ieee.numeric_std.ALL;
-- FPGA projects using VHDL/ VHDL
-- fpga4student.com
-- VHDL code for FIFO memory
-- Memory array
entity memory_array is
   port(
      data_out : out std_logic_vector(7 downto 0);
   rptr: in  std_logic_vector(4 downto 0);
      clk :in std_logic;
   fifo_we: in std_logic;
      wptr :in  std_logic_vector(4 downto 0);
   data_in: in std_logic_vector(7 downto 0)
   );
end memory_array;
architecture Behavioral of memory_array is
 type mem_array is array (0 to 15) of std_logic_vector(7 downto 0);
 signal data_out2: mem_array;
begin

 process(clk)
 begin
     if(rising_edge(clk)) then
   if(fifo_we='1') then
          data_out2(to_integer(unsigned(wptr(3 downto 0)))) <= data_in;
   end if;
  end if;
 end process;
 data_out <= data_out2(to_integer(unsigned(rptr(3 downto 0))));

end Behavioral;

Library IEEE;
USE IEEE.Std_logic_1164.all;
USE ieee.std_logic_arith.all;
USE ieee.std_logic_unsigned.all;
 -- FPGA projects using VHDL/ VHDL
-- fpga4student.com
-- VHDL code for FIFO memory
-- Read pointer
entity read_pointer is
   port(
      rptr : out std_logic_vector(4 downto 0);
   fifo_rd: out std_logic;
      clk :in std_logic;
   rst_n: in std_logic;
      rd :in  std_logic;
   fifo_empty: in std_logic
   );
end read_pointer;
architecture Behavioral of read_pointer is
signal re: std_logic;
signal read_addr:std_logic_vector(4 downto 0);
begin
 rptr <= read_addr;
 fifo_rd <= re ;
 re <= (not fifo_empty) and rd;
 process(clk,rst_n)
```

```
 begin
     if(rst_n='0') then
   read_addr <= (others => '0');
     elsif(rising_edge(clk)) then
   if(re='1') then
    read_addr <= read_addr + "00001";
   end if;
  end if;
 end process;
end Behavioral;

Library IEEE;
USE IEEE.Std_logic_1164.all;
USE ieee.std_logic_arith.all;
USE ieee.std_logic_unsigned.all;
-- FPGA projects using VHDL/ VHDL
-- fpga4student.com
-- VHDL code for FIFO memory
-- Write pointer
entity write_pointer is
   port(
      wptr : out std_logic_vector(4 downto 0);
   fifo_we: out std_logic;
      clk :in std_logic;
   rst_n: in std_logic;
      wr :in  std_logic;
   fifo_full: in std_logic
   );
end write_pointer;
architecture Behavioral of write_pointer is
signal we: std_logic;
signal write_addr:std_logic_vector(4 downto 0);
begin
 fifo_we <= we;
 we <= (not fifo_full) and wr;
 wptr <= write_addr;
 process(clk,rst_n)
 begin
     if(rst_n='0') then
   write_addr <= (others => '0');
     elsif(rising_edge(clk)) then
   if(we='1') then
    write_addr <= write_addr + "00001";
   end if;
  end if;
 end process;
end Behavioral;
```

The VHDL code for FIFO memory is verified by using the same testbench code as the previous post. I used Xilinx ISIM to run mixed language simulation. The Verilog testbench code for the VHDL code of the FIFO memory, you can also download here.

After running simulation, the correct result should be shown as follows:

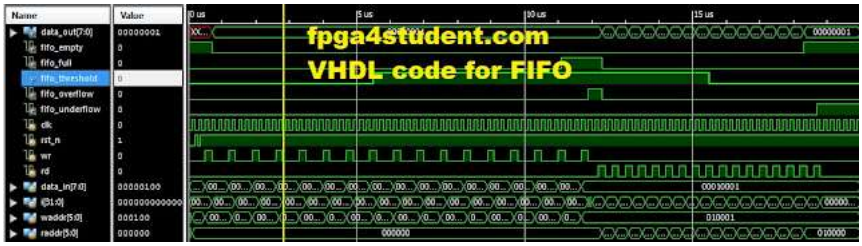
```
TIME =          110, data_out =   1, mem =   1
TIME =          120, wr = 1, rd = 0, data_in = 02
TIME =          130, data_out =   1, mem =   1
TIME =          140, wr = 0, rd = 0, data_in = 02
TIME =          150, data_out =   1, mem =   1
TIME =          170, data_out =   1, mem =   1
TIME =          190, data_out =   1, mem =   1
TIME =          190, wr = 1, rd = 0, data_in = 03
TIME =          210, data_out =   1, mem =   1
TIME =          210, wr = 0, rd = 0, data_in = 03
```

```
TIME =          230, data_out =   1, mem =   1
TIME =          250, data_out =   1, mem =   1
TIME =          260, wr = 1, rd = 0, data_in = 04
TIME =          270, data_out =   1, mem =   1
TIME =          280, wr = 0, rd = 0, data_in = 04
TIME =          290, data_out =   1, mem =   1
TIME =          310, data_out =   1, mem =   1
TIME =          330, data_out =   1, mem =   1
TIME =          330, wr = 1, rd = 0, data_in = 05
TIME =          350, data_out =   1, mem =   1
TIME =          350, wr = 0, rd = 0, data_in = 05
TIME =          370, data_out =   1, mem =   1
TIME =          390, data_out =   1, mem =   1
TIME =          400, wr = 1, rd = 0, data_in = 06
TIME =          410, data_out =   1, mem =   1
TIME =          420, wr = 0, rd = 0, data_in = 06
TIME =          430, data_out =   1, mem =   1
TIME =          450, data_out =   1, mem =   1
TIME =          470, data_out =   1, mem =   1
TIME =          470, wr = 1, rd = 0, data_in = 07
TIME =          490, data_out =   1, mem =   1
TIME =          490, wr = 0, rd = 0, data_in = 07
TIME =          510, data_out =   1, mem =   1
TIME =          530, data_out =   1, mem =   1
TIME =          540, wr = 1, rd = 0, data_in = 08
TIME =          550, data_out =   1, mem =   1
TIME =          560, wr = 0, rd = 0, data_in = 08
TIME =          570, data_out =   1, mem =   1
TIME =          590, data_out =   1, mem =   1
TIME =          610, data_out =   1, mem =   1
TIME =          610, wr = 1, rd = 0, data_in = 09
TIME =          630, data_out =   1, mem =   1
TIME =          630, wr = 0, rd = 0, data_in = 09
TIME =          650, data_out =   1, mem =   1
TIME =          670, data_out =   1, mem =   1
TIME =          680, wr = 1, rd = 0, data_in = 0a
TIME =          690, data_out =   1, mem =   1
TIME =          700, wr = 0, rd = 0, data_in = 0a
TIME =          710, data_out =   1, mem =   1
TIME =          730, data_out =   1, mem =   1
TIME =          750, data_out =   1, mem =   1
TIME =          750, wr = 1, rd = 0, data_in = 0b
TIME =          770, data_out =   1, mem =   1
TIME =          770, wr = 0, rd = 0, data_in = 0b
TIME =          790, data_out =   1, mem =   1
TIME =          810, data_out =   1, mem =   1
TIME =          820, wr = 1, rd = 0, data_in = 0c
TIME =          830, data_out =   1, mem =   1
TIME =          840, wr = 0, rd = 0, data_in = 0c
TIME =          850, data_out =   1, mem =   1
TIME =          870, data_out =   1, mem =   1
TIME =          890, data_out =   1, mem =   1
TIME =          890, wr = 1, rd = 0, data_in = 0d
TIME =          910, data_out =   1, mem =   1
TIME =          910, wr = 0, rd = 0, data_in = 0d
TIME =          930, data_out =   1, mem =   1
TIME =          950, data_out =   1, mem =   1
TIME =          960, wr = 1, rd = 0, data_in = 0e
```

```
TIME =          970, data_out =   1, mem =   1
TIME =          980, wr = 0, rd = 0, data_in = 0e
TIME =          990, data_out =   1, mem =   1
TIME =         1010, data_out =   1, mem =   1
TIME =         1030, data_out =   1, mem =   1
TIME =         1030, wr = 1, rd = 0, data_in = 0f
TIME =         1050, data_out =   1, mem =   1
TIME =         1050, wr = 0, rd = 0, data_in = 0f
TIME =         1070, data_out =   1, mem =   1
TIME =         1090, data_out =   1, mem =   1
TIME =         1100, wr = 1, rd = 0, data_in = 10
TIME =         1110, data_out =   1, mem =   1
TIME =         1120, wr = 0, rd = 0, data_in = 10
TIME =         1130, data_out =   1, mem =   1
TIME =         1150, data_out =   1, mem =   1
TIME =         1170, data_out =   1, mem =   1
TIME =         1170, wr = 1, rd = 0, data_in = 11
TIME =         1190, data_out =   1, mem =   1
TIME =         1190, wr = 0, rd = 0, data_in = 11
TIME =         1210, data_out =   1, mem =   1
TIME =         1220, wr = 0, rd = 1, data_in = 11
TIME =         1230, data_out =   1, mem =   1
PASS ------ PASS ---------- PASS ---------- PASS
TIME =         1240, wr = 0, rd = 0, data_in = 11
TIME =         1250, data_out =   2, mem =   2
TIME =         1260, wr = 0, rd = 1, data_in = 11
TIME =         1270, data_out =   2, mem =   2
PASS ------ PASS ---------- PASS ---------- PASS
TIME =         1280, wr = 0, rd = 0, data_in = 11
TIME =         1290, data_out =   3, mem =   3
TIME =         1300, wr = 0, rd = 1, data_in = 11
TIME =         1310, data_out =   3, mem =   3
PASS ------ PASS ---------- PASS ---------- PASS
TIME =         1320, wr = 0, rd = 0, data_in = 11
TIME =         1330, data_out =   4, mem =   4
TIME =         1340, wr = 0, rd = 1, data_in = 11
TIME =         1350, data_out =   4, mem =   4
PASS ------ PASS ---------- PASS ---------- PASS
TIME =         1360, wr = 0, rd = 0, data_in = 11
TIME =         1370, data_out =   5, mem =   5
TIME =         1380, wr = 0, rd = 1, data_in = 11
TIME =         1390, data_out =   5, mem =   5
PASS ------ PASS ---------- PASS ---------- PASS
TIME =         1400, wr = 0, rd = 0, data_in = 11
TIME =         1410, data_out =   6, mem =   6
TIME =         1420, wr = 0, rd = 1, data_in = 11
TIME =         1430, data_out =   6, mem =   6
PASS ------ PASS ---------- PASS ---------- PASS
TIME =         1440, wr = 0, rd = 0, data_in = 11
TIME =         1450, data_out =   7, mem =   7
TIME =         1460, wr = 0, rd = 1, data_in = 11
TIME =         1470, data_out =   7, mem =   7
PASS ------ PASS ---------- PASS ---------- PASS
TIME =         1480, wr = 0, rd = 0, data_in = 11
TIME =         1490, data_out =   8, mem =   8
TIME =         1500, wr = 0, rd = 1, data_in = 11
TIME =         1510, data_out =   8, mem =   8
PASS ------ PASS ---------- PASS ---------- PASS
```

```
TIME =          1520, wr = 0, rd = 0, data_in = 11
TIME =          1530, data_out =  9, mem =  9
TIME =          1540, wr = 0, rd = 1, data_in = 11
TIME =          1550, data_out =  9, mem =  9
PASS ------ PASS ---------- PASS -------------- PASS
TIME =          1560, wr = 0, rd = 0, data_in = 11
TIME =          1570, data_out = 10, mem = 10
TIME =          1580, wr = 0, rd = 1, data_in = 11
TIME =          1590, data_out = 10, mem = 10
PASS ------ PASS ---------- PASS -------------- PASS
TIME =          1600, wr = 0, rd = 0, data_in = 11
TIME =          1610, data_out = 11, mem = 11
TIME =          1620, wr = 0, rd = 1, data_in = 11
TIME =          1630, data_out = 11, mem = 11
PASS ------ PASS ---------- PASS -------------- PASS
TIME =          1640, wr = 0, rd = 0, data_in = 11
TIME =          1650, data_out = 12, mem = 12
TIME =          1660, wr = 0, rd = 1, data_in = 11
TIME =          1670, data_out = 12, mem = 12
PASS ------ PASS ---------- PASS -------------- PASS
TIME =          1680, wr = 0, rd = 0, data_in = 11
TIME =          1690, data_out = 13, mem = 13
TIME =          1700, wr = 0, rd = 1, data_in = 11
TIME =          1710, data_out = 13, mem = 13
PASS ------ PASS ---------- PASS -------------- PASS
TIME =          1720, wr = 0, rd = 0, data_in = 11
TIME =          1730, data_out = 14, mem = 14
TIME =          1740, wr = 0, rd = 1, data_in = 11
TIME =          1750, data_out = 14, mem = 14
PASS ------ PASS ---------- PASS -------------- PASS
TIME =          1760, wr = 0, rd = 0, data_in = 11
TIME =          1770, data_out = 15, mem = 15
TIME =          1780, wr = 0, rd = 1, data_in = 11
TIME =          1790, data_out = 15, mem = 15
PASS ------ PASS ---------- PASS -------------- PASS
TIME =          1800, wr = 0, rd = 0, data_in = 11
TIME =          1810, data_out = 16, mem = 16
TIME =          1820, wr = 0, rd = 1, data_in = 11
TIME =          1830, data_out = 16, mem = 16
PASS ------ PASS ---------- PASS -------------- PASS
```

Simulation waveform for the FIFO memory:



By observing the simulation waveform and memory, it is easy to see how data is written into the FIFO and how data is read out from the FIFO. It is noticed that the status signals such as Overflow, Underflow, Empty, Full are very critical to determine the correctness of the FIFO.

Recommended VHDL projects:

1. **What is an FPGA? How VHDL works on FPGA**
2. **VHDL code for FIFO memory**
3. **VHDL code for FIR Filter**
4. **VHDL code for 8-bit Microcontroller**
5. **VHDL code for Matrix Multiplication**

6. VHDL code for Switch Tail Ring Counter
7. VHDL code for digital alarm clock on FPGA
8. VHDL code for 8-bit Comparator
9. How to load a text file into FPGA using VHDL
10. VHDL code for D Flip Flop
11. VHDL code for Full Adder
12. PWM Generator in VHDL with Variable Duty Cycle
13. VHDL code for ALU
14. VHDL code for counters with testbench
15. VHDL code for 16-bit ALU
16. Shifter Design in VHDL
17. Non-linear Lookup Table Implementation in VHDL
18. Cryptographic Coprocessor Design in VHDL
19. Verilog vs VHDL: Explain by Examples
20. VHDL Code for Clock Divider on FPGA
21. How to generate a clock enable signal instead of creating another clock domain
22. VHDL code for debouncing buttons on FPGA
23. VHDL code for Traffic light controller
24. VHDL code for a simple 2-bit comparator
25. VHDL code for a single-port RAM
22. VHDL code for debouncing buttons on FPGA
23. VHDL code for Traffic light controller
24. VHDL code for a simple 2-bit comparator
25. VHDL code for a single-port RAM
26. VHDL code for Car Parking System using FSM
27. VHDL coding vs Software Programming
28. VHDL code for MIPS Processor
29. VHDL code for Moore FSM Sequence Detector
30. VHDL code for Seven-Segment Display on Basys 3 FPGA

DMCA PROTECTED

Privacy Policy | Disclaimer | Sitemap | Contact | Advertise Here
Copyright © 2016-2020 FPGA4student.com All Rights Reserved.